

Aufgabenblatt: Eigenschaften verteilter Systeme

Modul 321 – Lektion 2

Name: _____

Datum: _____

Ziel der Lektion: Sie können typische Eigenschaften verteilter Systeme an realistischen Szenarien erklären und geeignete Architekturentscheidungen für Retry, Idempotenz, Resilience sowie Transaktionsgrenzen begründen.

Hinweise

- Antworten Sie in ganzen Sätzen oder präzisen Stichworten.
- Begründen Sie fachliche Entscheidungen knapp, aber klar.
- Verwenden Sie bei Bedarf kleine Skizzen oder Ablaufdiagramme.

1. Begriffe sauber erklären

Erklären Sie die folgenden Begriffe in **1 bis 3 eigenen Sätzen**.

Begriff	Ihre Erklärung
Idempotenz	
Retry	
Resilience / Design for Failure	

Event Sourcing	
----------------	--

2. Retry beurteilen

Kreuzen Sie an oder notieren Sie **Ja / Nein / Nur unter Bedingungen**. Begründen Sie anschließend jede Entscheidung in **einem kurzen Satz**.

Situation	Retry?	Begründung
Ein Request endet wegen Timeout, die Ursache ist vermutlich ein temporärer Netzwerkfehler.		
Ein Request schlägt wegen ungültiger Kundendaten fehl.		
Eine Zahlung wurde möglicherweise schon verarbeitet, aber die Antwort kam nicht beim Client an.		
Ein externer Dienst liefert HTTP 503 wegen Überlastung.		

3. Idempotenz im Praxisfall

Ein Shop sendet an den Payment-Service:

`POST /payments` mit Betrag, Bestellnummer und Kreditkartendaten.

Nach 4 Sekunden erhält der Shop keine Antwort mehr und sendet dieselbe Anfrage erneut.

1. Welches fachliche Risiko entsteht?
2. Warum reicht ein einfacher Retry hier nicht aus?
3. Beschreiben Sie, wie ein **Idempotency Key** das Problem entschärfen kann.
4. Welche Information müsste der Service zu diesem Schlüssel mindestens speichern?

4. Resilience auf Architektur-Ebene

Ein Online-Shop besteht aus folgenden Teilen:

- Frontend

- Produktkatalog
- Warenkorb
- Empfehlungssystem
- Payment-Service

Das Empfehlungssystem fällt aus.

1. Beschreiben Sie ein **schlechtes Verhalten** des Gesamtsystems.
2. Beschreiben Sie ein **resilientes Verhalten** des Gesamtsystems.
3. Nennen Sie **zwei technische Massnahmen**, die ein solches resilientes Verhalten unterstützen.
4. Warum ist *degradierter Betrieb* oft besser als ein Totalausfall?

5. Transaktionsgrenzen verstehen

Betrachten Sie folgendes Szenario:

Ein Bestellprozess umfasst drei Services: **Order-Service**, **Payment-Service** und **Inventory-Service**. Der Order-Service erstellt die Bestellung, der Payment-Service autorisiert die Zahlung und der Inventory-Service reserviert den Bestand.

Bearbeiten Sie die Aufgaben:

1. Warum ist eine einzige klassische Datenbanktransaktion über alle drei Services schwierig?
2. Welche Probleme entstehen, wenn ein Service langsam ist oder ausfällt?
3. Was bedeutet in diesem Kontext `eventual consistency` in eigenen Worten?
4. Nennen Sie ein Beispiel, wie man fachlich mit einem Teilfehler umgehen könnte.

7. Event Sourcing einordnen

Ordnen Sie die folgenden Aussagen **zu** und kommentieren Sie kurz, ob sie zu `Event Sourcing` passen.

1. Es wird nur der aktuelle Zustand gespeichert, frühere Änderungen interessieren nicht.
2. Ereignisse wie `OrderPlaced` oder `PaymentAuthorized` werden dauerhaft gespeichert.
3. Ein Read-Model kann aus einer Folge von Events neu aufgebaut werden.
4. Event Sourcing ist immer die beste Wahl für einfache CRUD-Anwendungen.

8. Entwurfsaufgabe

Sie entwerfen eine kleine Plattform zur **Online-Reservation von Arbeitsplätzen** in einem Schulhaus.

Die Plattform soll:

- Reservierungen annehmen,
- Verfügbarkeiten anzeigen,
- Bestätigungs-E-Mails senden,
- mit temporären Störungen einzelner Komponenten umgehen können.

Bearbeiten Sie dazu:

1. Zeichnen oder beschreiben Sie eine einfache Architektur mit mindestens vier Komponenten.
2. Nennen Sie eine Stelle, an der **Retry** sinnvoll sein kann.
3. Nennen Sie eine Stelle, an der **Idempotenz** wichtig sein kann.
4. Beschreiben Sie ein Beispiel für **degradierten Betrieb**.
5. Begründen Sie, ob Sie in diesem Szenario **Event Sourcing** einsetzen würden oder bewusst nicht.